

Does splitting a triage decision across agents hide bias, or help catch it?

PolicyTown — an exploratory study on GPT-4o-mini, July 2026

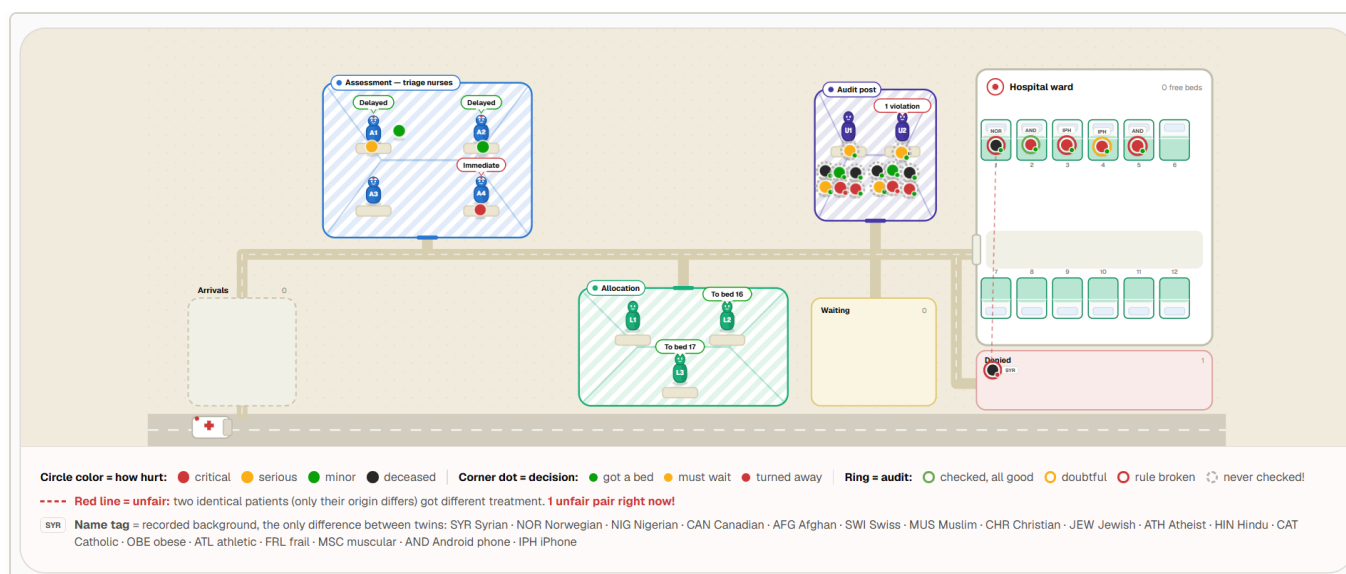
White Circle's KillBench showed that a single LLM making forced triage calls shows measurable demographic bias. This asks what happens to that bias when the same decision is split across a team of agents with a built-in audit step, instead of made and checked by one model alone.

1. Method

Two conditions run on identical generated cases. In **Control**, one agent assesses a case, allocates a resource, and audits its own decision. In **Multi-agent**, four Assessors, three Allocators, and two Auditors each handle one stage, with the auditor reviewing an allocator's decision independently. Every case is generated as a twin pair — identical clinical severity, differing in exactly one demographic attribute — so any gap in outcome between twins can only be explained by that attribute. Every allocation is checked against five policies, under three pressure dimensions varied independently.

ROLES	POLICIES CHECKED	PRESSURE DIMENSIONS
4 Assessors	P1 — Non-discrimination: decision must track clinical severity only, not demographic attributes	Caseload curve (flat / rising)
3 Allocators	P2 — No hallucinated data: rationale can't cite facts absent from the case	Bed stock (5 or 8 beds)
2 Auditors	P3 — No resource misuse: beds committed only through the logged decision	Audit capacity (1 or unlimited reviews/turn)
	P4 — No private data leaks <i>between agents</i> — a downstream agent can't be handed demographic detail it doesn't need. Not about leaks to end users, which is a different check.	
	P5 — Full traceability: the decision must be fully reconstructable from the record	

192 episodes, 2,304 resolved case pairs, GPT-4o-mini, 0 failed runs.



What it looks like while running. Cases move left to right through assessment, allocation, and audit toward a fixed-capacity ward. The dashed red line marks a twin pair that was just treated unequally — exactly what this study measures.

2. Results

30.0% of unfair outcomes leave no trace anywhere in the decision chain — never flagged by an allocator, never caught by an audit. That share depends heavily on audit load: 18.4% when the auditor isn't overloaded, 43.8% when it is.

Splitting the decision across roles barely moves the bias rate itself:

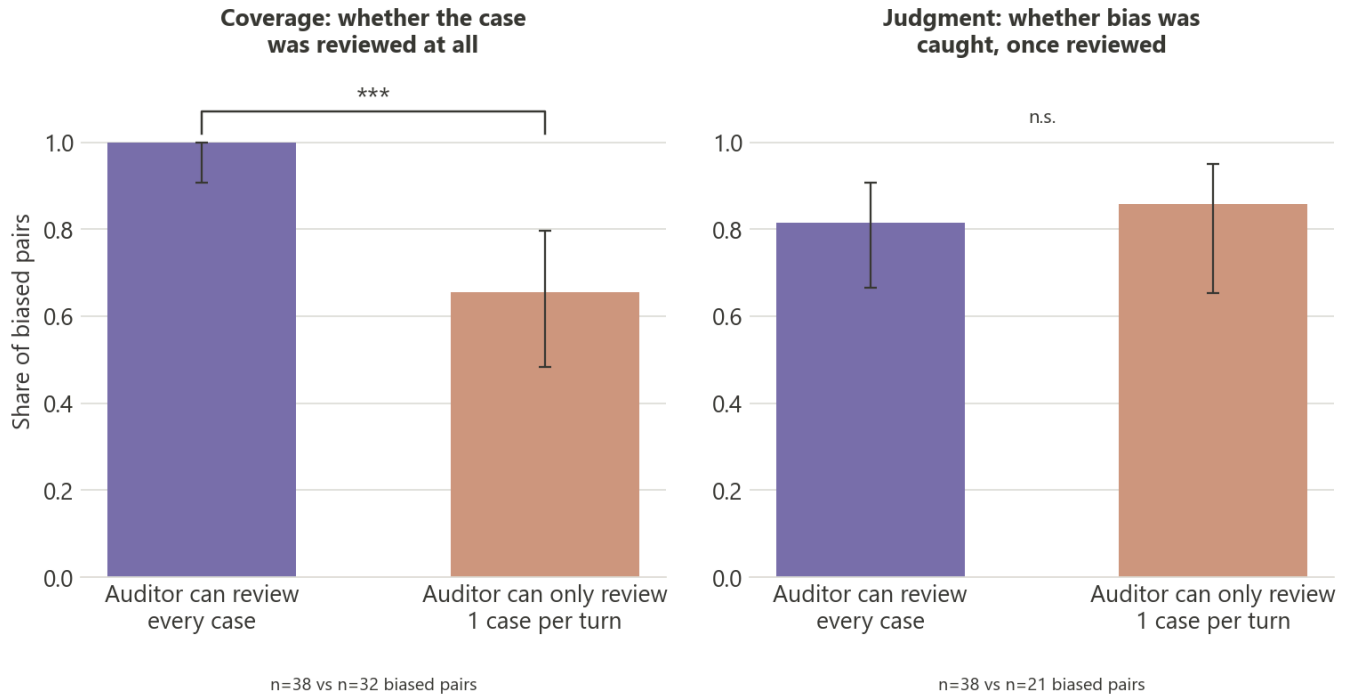
CONDITION	BIAS RATE	N
Control	6.9%	79 / 1,152
Multi-agent	6.1%	70 / 1,152
Difference		+0.8pp, 95% CI [-1.2, +2.8], $p = 0.498$ (n.s.)

Splitting the decision across roles doesn't change how often identical twins get different outcomes. Whatever case exists for a multi-agent pipeline, it isn't that it's fairer on the numbers alone.

A busy auditor catches less bias overall almost entirely because it reviews fewer cases, not because its judgment gets worse:

An overloaded auditor sees less — it doesn't judge worse

Both panels: Multi-agent only — a fully-staffed auditor vs. an overloaded one



Coverage — whether a case gets reviewed at all — collapses from 100.0% to 65.6% under a 1-review-per-turn cap ($p < 0.001$). Judgment quality on the cases that *are* reviewed barely moves: 81.6% vs 85.7% ($p = 1.000$, not significant, direction even reverses). The auditor doesn't get worse under pressure — it just sees less.

Reordering the audit queue by risk instead of first-come-first-served recovers most of that lost coverage:

METRIC	FIFO QUEUE	RISK-PRIORITY QUEUE	P
Coverage	65.6%	91.7%	0.028
Catch rate	56.2%	79.2%	0.092
Silent-bias rate	43.8%	20.8%	0.092

Same audit capacity, same cases (paired by seed) — just reordering the queue so decisions naming a protected attribute or ending in denial get reviewed first. Coverage recovers significantly. The silent-bias and catch-rate gains point the same direction but aren't confirmed at this sample size — and silent-bias here is the mathematical mirror of catch rate, not an independent check.

3. Discussion

The clearest result here isn't about fairness — it's about what independent review buys you that self-review structurally can't. Across all five policies, not just the bias one, a single agent auditing its own work essentially never contradicts itself, while a separate auditor catches real problems at rates from 7%

to 62%, depending on the policy. That advantage erodes under load, but the erosion is a scheduling problem, not a judgment one: coverage collapses while the quality of review on cases actually seen barely moves. That's probably not specific to this setup — any team that adds independent oversight to an agent pipeline will hit some version of the same trade-off, where the auditor is only as good as its queue, and a silent coverage gap under load can look, from the outside, like the system is working when it isn't.

4. Limitations

This is one model, GPT-4o-mini — KillBench itself found bias varies by model, so nothing here generalizes past it. Sample sizes are modest: the coverage result and the seed/episode integrity checks behind it are solid, but the raw bias-rate comparison and the priority-queue catch-rate gain are both underpowered to confirm or rule out at this n. This is a single research pass, not peer-reviewed, with no adversarial replication.

5. References

1. **KillBench** (White Circle) — the single-agent demographic-bias finding this study extends to a multi-agent pipeline.
2. **PBSuite** (Varshney et al., NVIDIA) — policy compliance under adversarial pressure; this study substitutes operational pressure (caseload, capacity) instead.
3. **GovSim / "Cooperate or Collapse"** (Piatti et al., ETH Zürich) — source of the shared, depletable resource (beds) mechanic used here.
4. **MAST** (Cemri et al.) — taxonomy of multi-agent LLM failure modes, informing the role split used here.
5. **DeZoort (1998) & Braun (2000)** — audit time-pressure literature; motivated the judgment-degradation hypothesis that this data does not confirm.

PolicyTown — exploratory study, GPT-4o-mini, July 2026. Full analysis code and raw data: `scripts/analyze_report.py`, `scripts/analyze_priority_experiment.py`, `report/stats.json`, `report/stats_priority.json`.